

DLD Lab 06

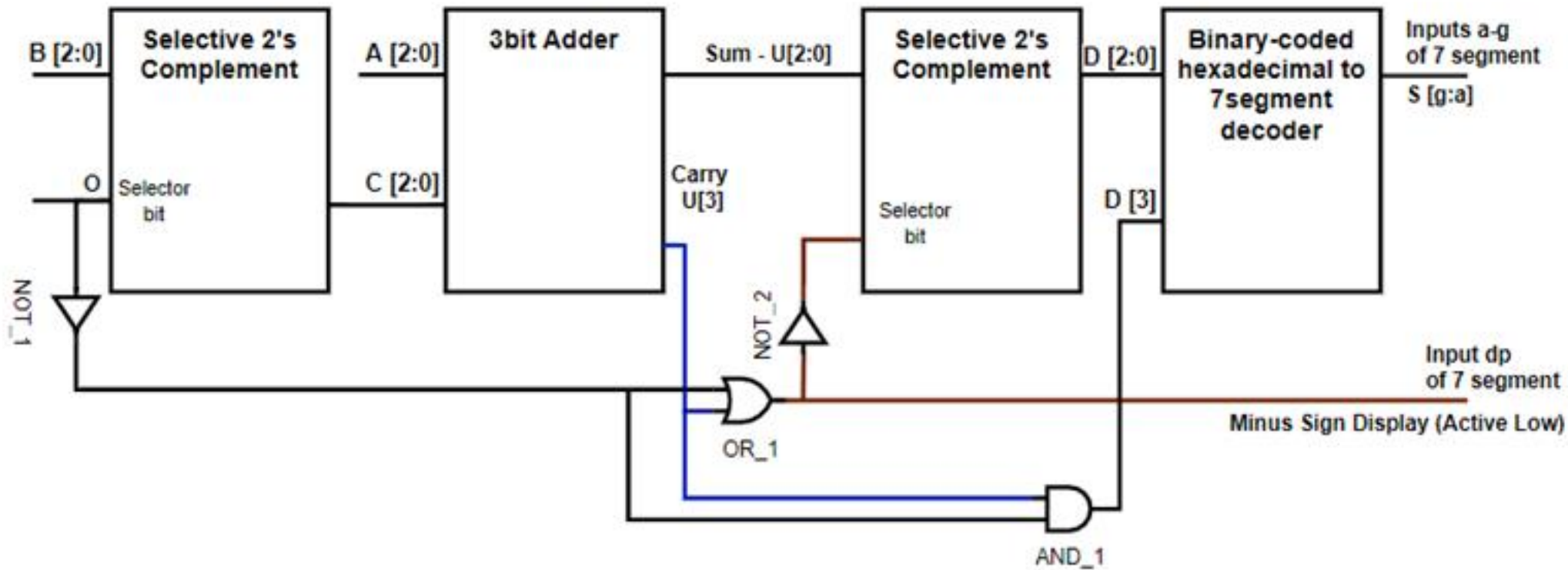
Seven Segment Decoder Design

Add/Subtract Module

Lab 06, 07, 08

1 = Subtraction || Find 2's complement
0 = Addition || Pass through

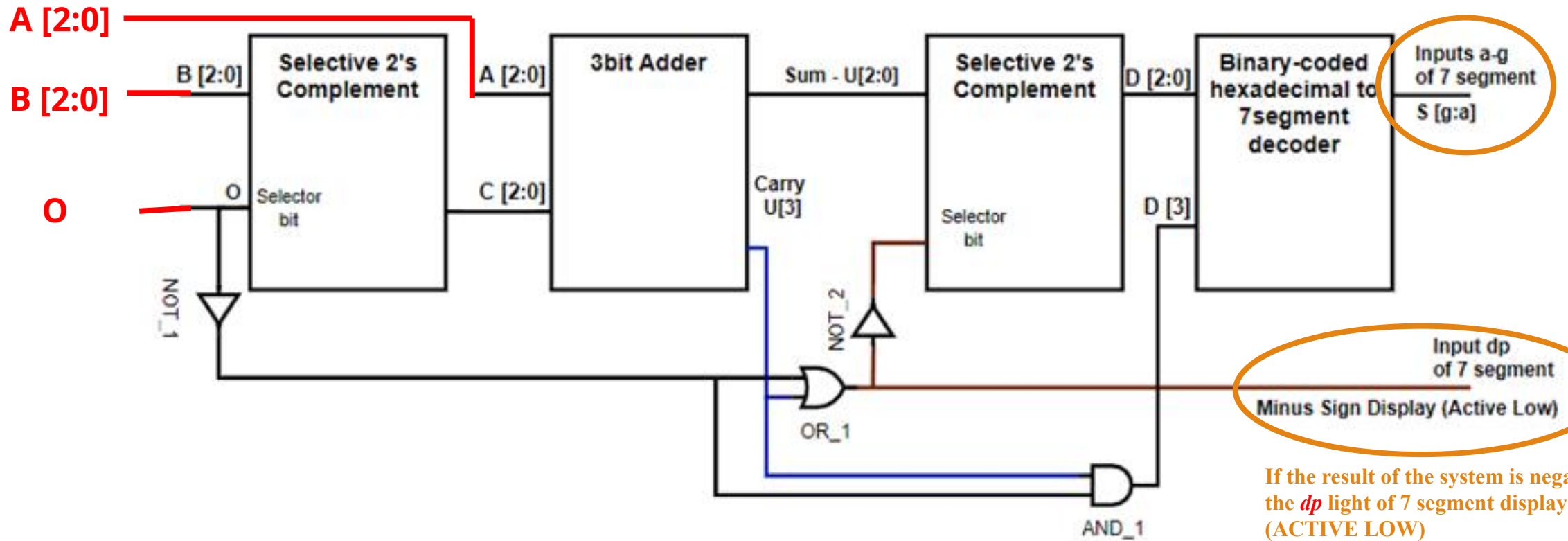
- System adds/subtracts two 3-bit binary numbers **A & B**.
- **O** is another 1 bit input that determines if the system will add/subtract.



Add/Subtract Module

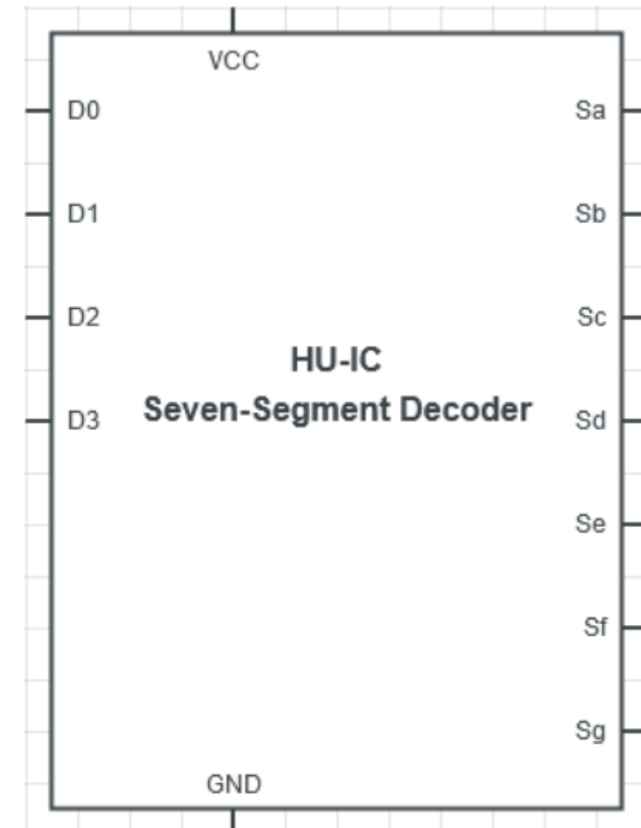
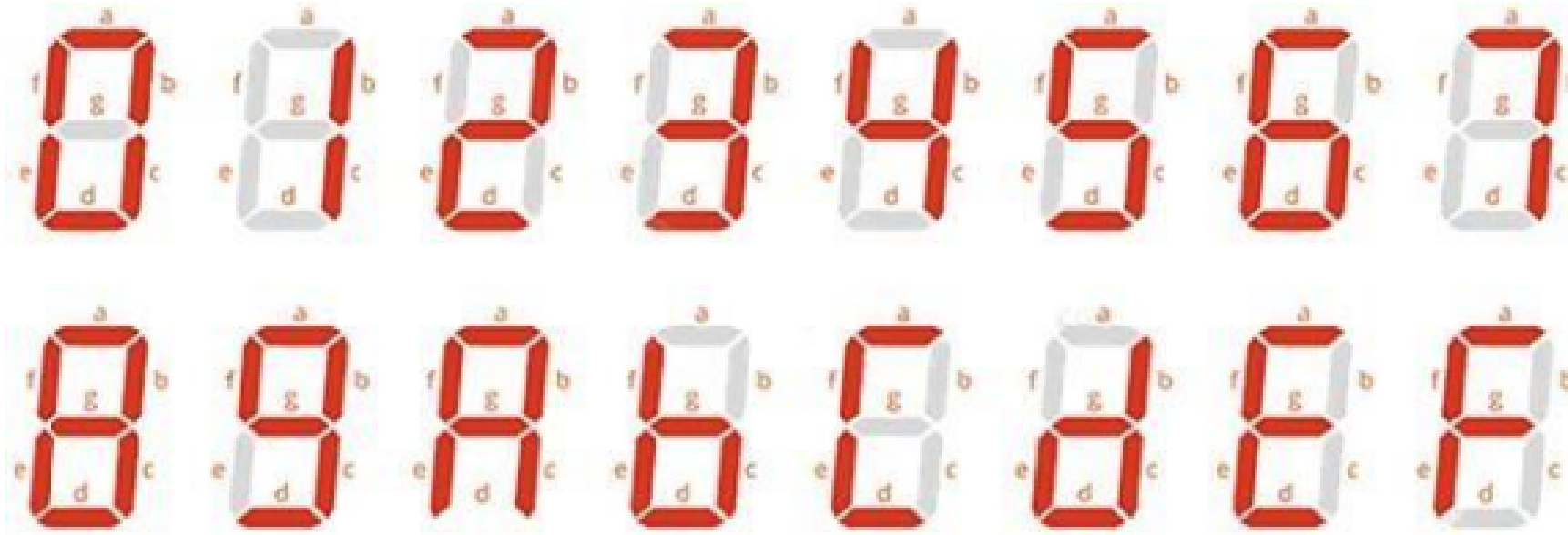
1 = Subtraction || Find 2's complement
0 = Addition || Pass through

- System generates the appropriate signals for 7 segment display to show the result of addition/subtraction as a hexadecimal digit.
- Common Anode (Active Low)



If the result of the system is negative then the *dp* light of 7 segment display lights up (ACTIVE LOW)

Seven Segment Display



- 7 segment decoder module replaces 7447 IC that takes 4 inputs (BCD) and decodes it into seven outputs.
- Decoder takes 4 input bits (D3-D0) representing binary coded hexadecimal number between 0 & F, and produces seven output bit s(Sg-Sa) – lights up appropriate segments of 7 segment display.

Seven Segment Display

Common Anode – Active Low

0 0 0 1
D₃ D₂ D₁ D₀



1 1 1 1 0 0 1
g f e d c b a



1 0 1 0
D₃ D₂ D₁ D₀

0 0 0 1 0 0 0
g f e d c b a

Example 1: Add 2 and 3 in decimal and display its result on seven segment display.

Operation: 2 + 3 = 5

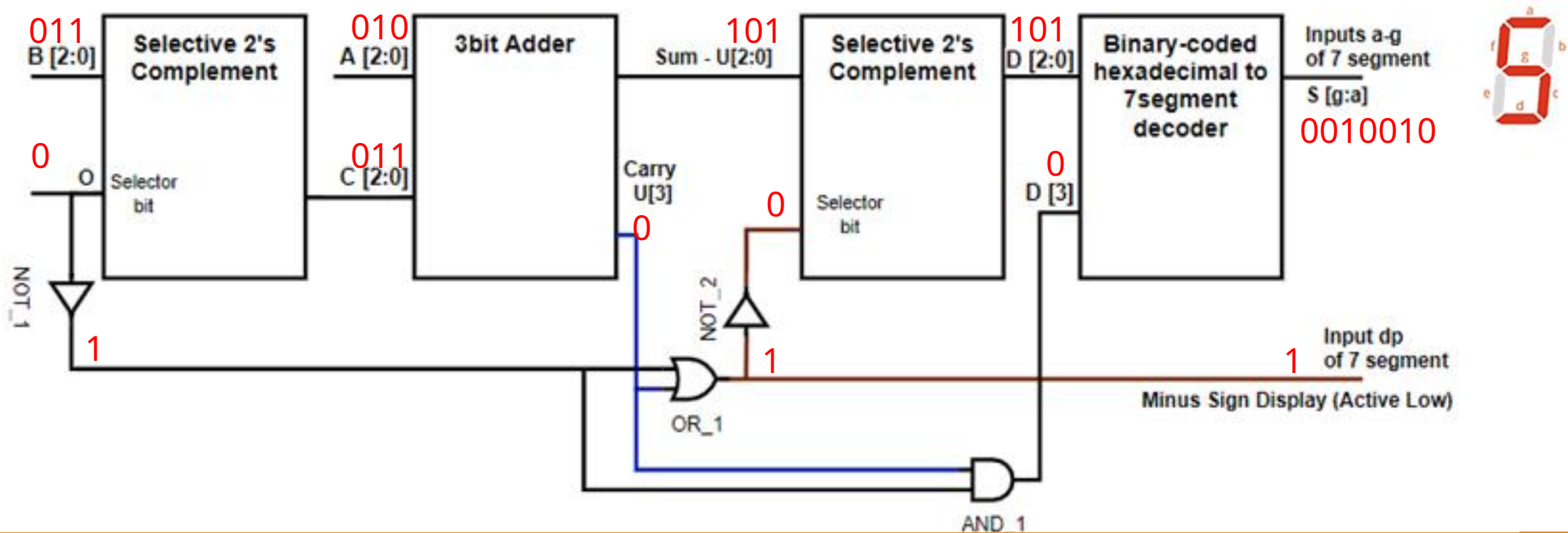
A = 3'b010

B = 3'b011

O = 0

1 = Subtraction || Find 2's complement
0 = Addition || Pass through

Hex Digit	Binary-coded Hexadecimal/ Inputs to decoder				Outputs of the decoder						
	D ₃	D ₂	D ₁	D ₀	S _g	S _r	S _e	S _d	S _c	S _b	S _a
0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	1	1	1	1	1	0	0	1
2	0	0	1	0	0	1	0	0	1	0	0
3	0	0	1	1	0	1	1	0	0	0	0
4	0	1	0	0	0	0	1	1	0	0	1
5	0	1	0	1	0	0	1	0	0	1	0
6	0	1	1	0	0	0	0	0	0	1	0



Example 2: Subtract 2 from 3 in decimal and display its result on seven segment display.

Operation: 3 – 2 = 1

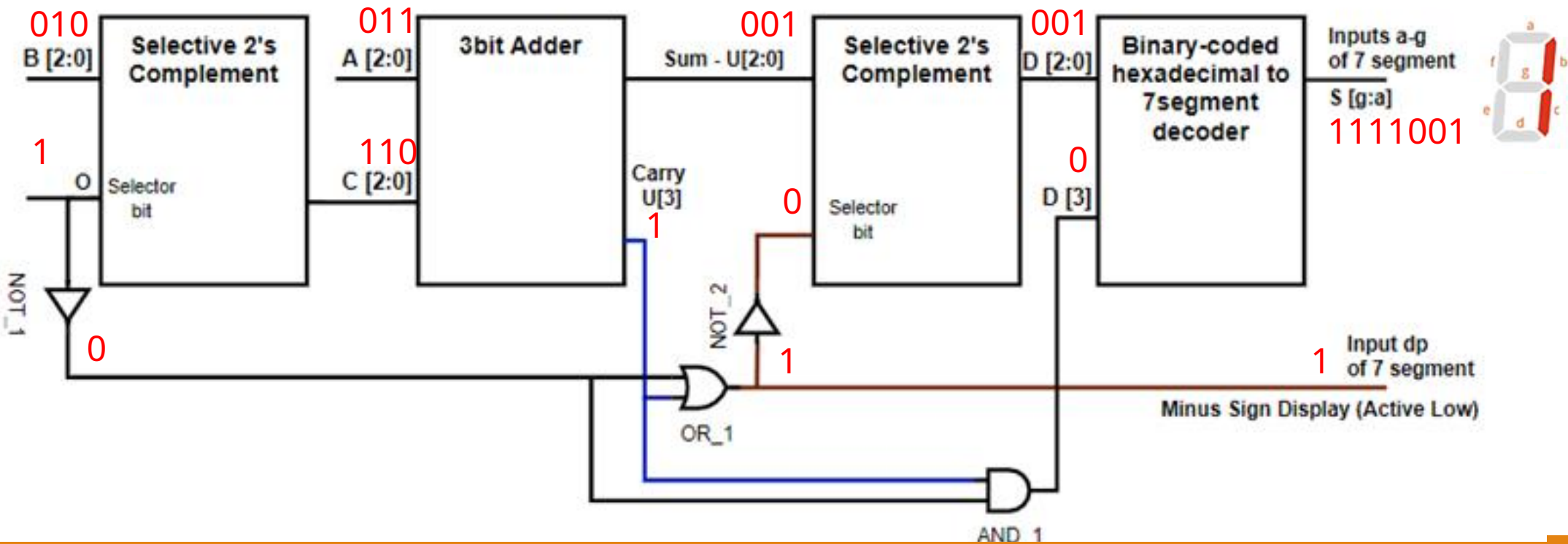
A = 3'b011

B = 3'b010

O = 1

1 = Subtraction || Find 2's complement
0 = Addition || Pass through

Hex Digit	Binary-coded Hexadecimal/ Inputs to decoder				Outputs of the decoder						
	D ₃	D ₂	D ₁	D ₀	S _g	S _r	S _e	S _d	S _c	S _b	S _a
0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	1	1	1	1	1	0	0	1
2	0	0	1	0	0	1	0	0	1	0	0
3	0	0	1	1	0	1	1	0	0	0	0
4	0	1	0	0	0	0	1	1	0	0	1
5	0	1	0	1	0	0	1	0	0	1	0
6	0	1	1	0	0	0	0	0	0	1	0



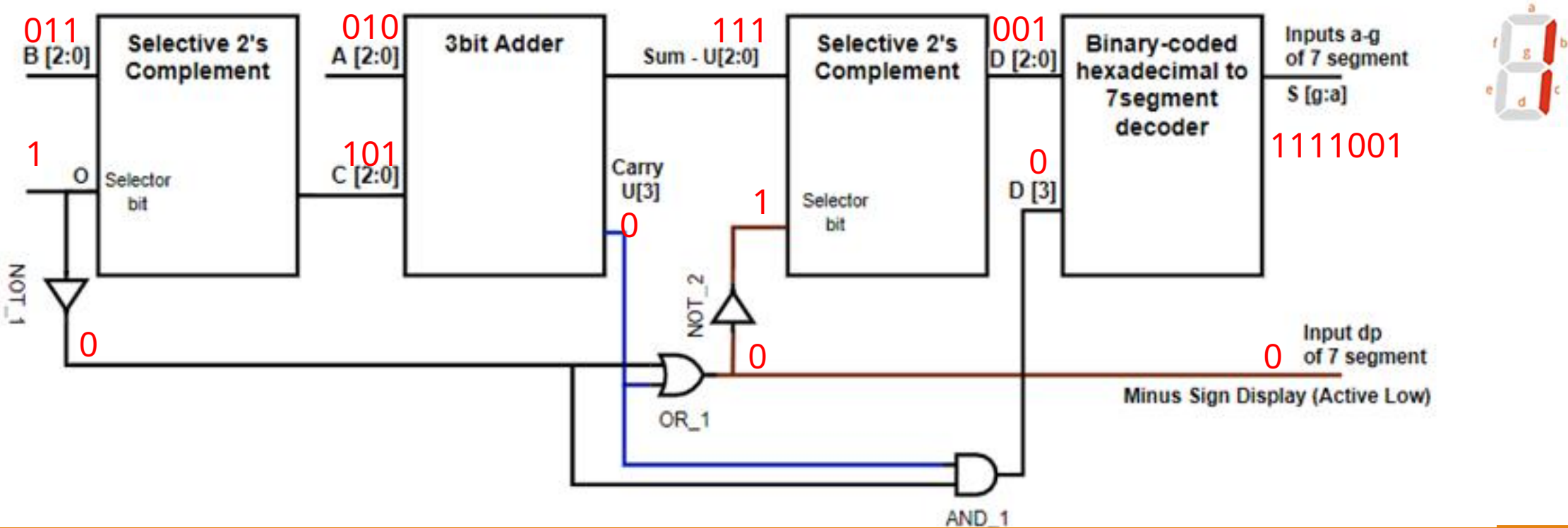
Example 3: Subtract 3 from 2 in decimal and display its result on seven segment display.

Operation: 2 – 3 = -1

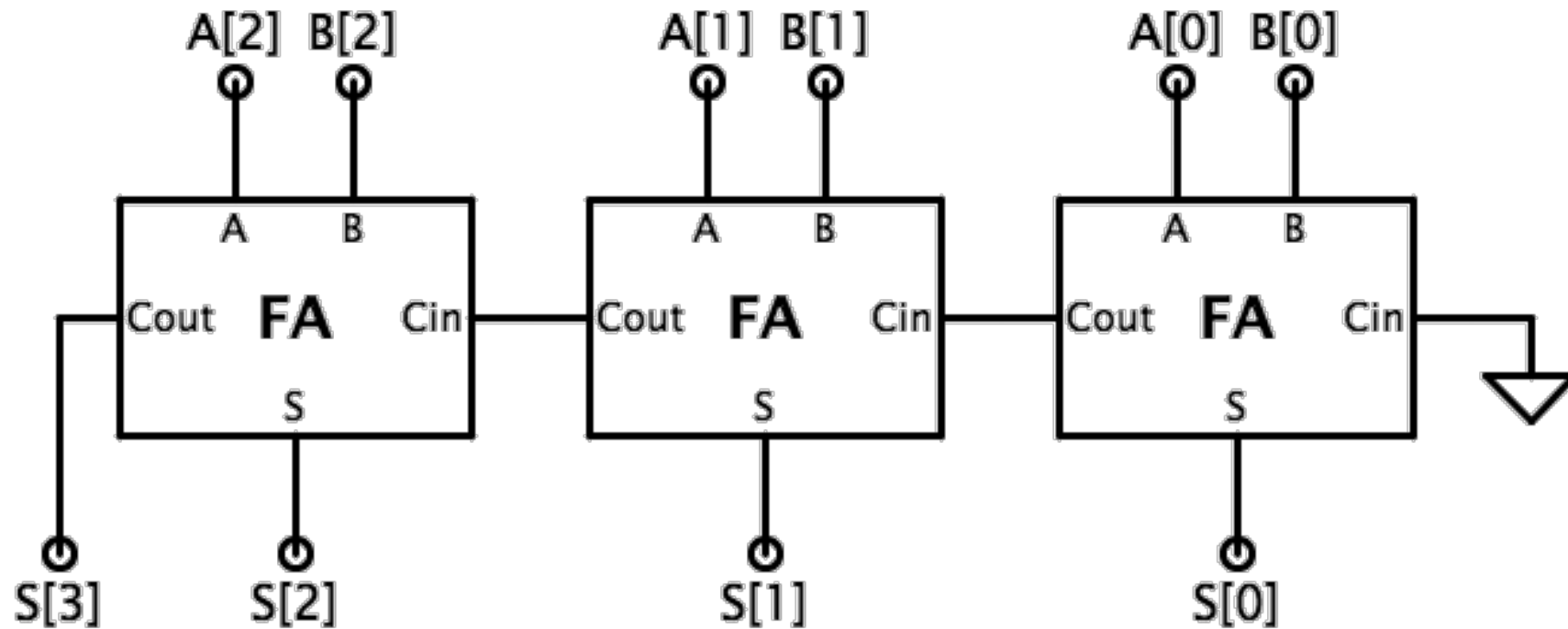
A = 3'b010
B = 3'b011
O = 1

1 = Subtraction || Find 2's complement
0 = Addition || Pass through

Hex Digit	Binary-coded Hexadecimal/ Inputs to decoder				Outputs of the decoder						
	D ₃	D ₂	D ₁	D ₀	S _g	S _f	S _e	S _d	S _c	S _b	S _a
0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	1	1	1	1	1	0	0	1
2	0	0	1	0	0	1	0	0	1	0	0
3	0	0	1	1	0	1	1	0	0	0	0
4	0	1	0	0	0	0	1	1	0	0	1
5	0	1	0	1	0	0	1	0	0	1	0
6	0	1	1	0	0	0	0	0	0	1	0

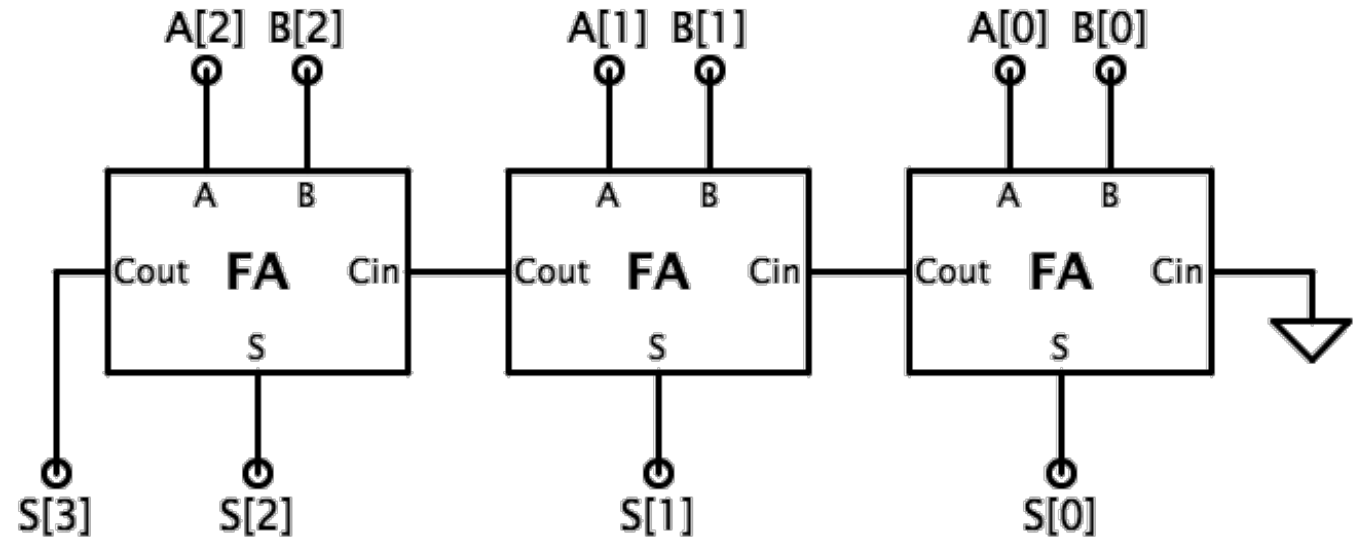


Task c: 3 bits Full Adder



Task c: 3 bits Full Adder

- Use the code provided in manual.
- Create 2 separate design files, FA using HA and 3-bit full adder using previous modules.



Task Guidelines



Task A. The similar test cases performed earlier is your task a (try a subtraction case with a negative output) (15 minutes)

Task B. Write code for 7 segment decoder

- Fill out the truth table (15 minutes)
- Use online k-map generator to find Boolean expression (20 minutes)
- Module & Testbench (40 minutes)

Task C. Create a 3 bits Full Adder module

- Use provided code to write full adder code – test if you wish. Testbench is not required
- Cascade 3 Full Adders to design a 3-bit Full Adder and test your module (30 minutes) – code is given
- Test cases: 2+1, 3+7, 3+3

Both of these codes from Task b & c will be used in Lab 07 & 08!

Common Errors in Vivado

Simulation – error

- ❖ Close previous simulation and re-run
- ❖ Same step for the schematic

Testing multiple testbenches

- ❖ Right click the test module (you intend to simulate) click on Set as top and then Run Simulation

Always check ‘Message’ tab for any error in the code

K-MAP Simplifier

- Choose the number of variable based on the number of inputs.
- For **each segment output (Sa through Sg)** choose the signal (0 or 1) for all possible combinations of inputs >> $A, B, C, D = D3, D2, D1, D0$.
- Create Boolean Expression for Sa through Sg outputs
- Notation in Verilog:
 $D3' D2' D1' D0 = (\sim D[3] \& \sim D[2] \& \sim D[1] \& D[0])$

Or = | **not** +

Not = ~

And = &

Hex Digit	Binary-coded Hexadecimal/ Inputs to decoder				Outputs of the decoder						
	D ₃	D ₂	D ₁	D ₀	S _g	S _f	S _e	S _d	S _c	S _b	S _a
0	0	0	0	0	1	0	0	0	0	0	0
1	0	0	0	1	1	1	1	1	0	0	1
2	0	0	1	0	0	1	0	0	1	0	0
3	0	0	1	1	0	1	1	0	0	0	0
4	0	1	0	0	0	0	1	1	0	0	1
5	0	1	0	1	0	0	1	0	0	1	0
6	0	1	1	0	0	0	0	0	0	1	0

[Link:](#)

[K-map simplifier](#)

D3 D2 D1 D0
A B C D

Announcements

- Team Formation – Number of groups in the section T2 = 7 at max (4-5 students each)
- In case anyone has not joined the group by tonight – will be randomly enrolled in a group (which has less than 4 members)
- Project Proposal Template & Sample will be released in the following week ON LMS